REQUIREMENTS ANALYSIS DOCUMENT

for

Automated Gherkin & BDD Test Scenario Generator

Issue 1.0 (Project Deliverable #1)

30 October 2026

Prepared by Team CURATORES

Emre Kaan ŞAHİN - 220104004047

Halit Batuhan ASLAN - 220104004003

Ömer Faruk GEDİKLİ - 220104004103

CSE 443 Object-Oriented Analysis and Design - Gebze Technical University - Fall 2026

Revision history

Issue	Date	Description	Authors
0.1	25 Oct 2026	First draft: scope, functional requirements, use cases, domain model	Team CURATORES
1.0	30 Oct 2026	Scope aligned with the instructor's answers (free text with LLM, optional GUI, negative and boundary scenarios, Background); data dictionary, Abbott analysis, five sequence diagrams, two state machines, GUI mock-ups and full traceability added	Team CURATORES

TABLE OF CONTENTS

1 Introduction 4
1.1 Purpose of the system 4
1.2 Scope of the system 4
1.3 Objectives and success criteria of the project 5
1.4 Definitions, acronyms, and abbreviations 6
1.5 References 6
1.6 Overview 7
2 Current system 8
3 Proposed system 9
3.1 Overview 9
3.2 Functional requirements 10
3.3 Nonfunctional requirements 14
3.3.1 Usability 15
3.3.2 Reliability 15
3.3.3 Performance 15
3.3.4 Supportability 15
3.3.5 Implementation 16
3.3.6 Interface 16
3.3.7 Packaging 16
3.3.8 Legal 17
3.4 System models 17
3.4.1 Scenarios 17
3.4.2 Use case model 19
3.4.3 Object model 23
3.4.4 Dynamic model 28
3.4.5 User interfaces and Mock-ups 34
4 Glossary of Terms 39
5 Priority 41
6 Traceability 42

6.1 Traceability to the project problem statement 42
6.2 Problems and objectives → use cases → requirements 42
6.3 Use case ↔ functional requirement matrix 42
6.4 Non-functional requirements → use cases 45
6.5 Requirements → domain classes 45
6.6 Summary 45

1 Introduction

This document is the Requirements Analysis Document (RAD) of the CSE 443 Object-Oriented Analysis and Design term project of Team CURATORES. It defines what the Automated Gherkin & BDD Test Scenario Generator (hereafter "the system") shall do and describes its analysis model with UML diagrams. The document follows the RAD template of the course [1] and the method of Bruegge and Dutoit [8]. Design decisions such as subsystem decomposition, class operations and the way the required design patterns (NFR-21) are applied are intentionally left to the System Design Document.

1.1 Purpose of the system

In software projects, acceptance test scenarios are usually written by hand from the requirements. We observed four problems with this practice:

- **P1 - Manual work is slow.** The test team writes the scenarios for every requirement one by one.
- **P2 - Results are inconsistent.** Different people interpret the same requirement in different ways and produce different scenarios.
- **P3 - Tests and requirements drift apart.** When a requirement changes, the related scenarios are often not updated, and nobody can see which requirement a scenario tests.
- **P4 - Negative and edge cases are forgotten.** Hand-written scenarios mostly cover the "happy path"; the cases where a condition is not met or a limit is reached are often missing.

The purpose of the system is to remove these problems. The system takes requirements written as EARS sentences or user stories (and, optionally, free text) and automatically produces Gherkin feature files with Given / When / Then scenarios. Because fixed mapping rules are applied every time, the same requirement always produces the same scenario; every scenario carries the ID of its requirement; and, on request, the system adds negative and boundary scenarios for each requirement.

1.2 Scope of the system

In scope

- Reading requirements from UTF-8 plain-text files (.txt, .md).
- Three input formats: EARS requirements, user stories ("As a … I want … so that …") with optional acceptance criteria, and free-text requirements.
- All five EARS patterns - ubiquitous, event-driven (When), state-driven (While), unwanted behaviour (If … then) and optional feature (Where) - and their combinations in one sentence.
- Validation of each requirement and an error report with line number, reason, expected pattern and a correction hint.
- Rule-based translation of EARS requirements and user stories into Gherkin Features, Backgrounds, Scenarios, Scenario Outlines and Steps.
- Optional generation of negative scenarios and boundary-value Scenario Outlines.
- Optional normalization of free-text requirements into EARS form by an external LLM Service, with acceptance by the Test Engineer.
- Preview of the generated scenarios and export as Gherkin (.feature), Markdown or JSON files.
- A command-line interface (CLI) as the main interface and a simple graphical interface (GUI) as an optional interface.

Out of scope

- Writing step definitions (glue code) or running the generated tests. The output is meant to be run by Cucumber [5] or a similar tool.
- Integration with requirement or test management tools (e.g., Jira) and version control systems.
- Requirements in languages other than English.
- Generating scenarios from complete use case documents.

Scope decisions agreed with the instructor

We presented the project and asked our open questions in a meeting with the instructor on 12 October 2026 [3]. Table 1 shows the answers and how they are reflected in this document.

Table 1: *Questions to the instructor and resulting decisions*

Question	Instructor's answer [3]	Decision in this document
Which EARS patterns must be supported?	In general all of them.	All five patterns and their combinations (FR-13).
Free text or structured input?	Free text is possible; the team decides.	EARS and user stories are parsed with rules (Must). Free text is accepted and normalized to EARS by an LLM Service (UC-9; the normalization itself has the priority Could, see Section 5).
Rule-based parsing or NLP / LLM?	An LLM may be used.	Rule-based core (NFR-20); the LLM is an optional, replaceable external actor (NFR-16).
Must the output run with Cucumber?	Not required, but it would be good.	Output follows the Cucumber Gherkin syntax (NFR-24).
Is a GUI required?	Not required; a GUI would be nice.	CLI is the main interface; a GUI is optional (NFR-28).
Positive scenarios only?	Edge cases, negative cases, performance and stress should also be considered; shared set-up can be grouped.	Negative scenarios (FR-28), boundary Scenario Outlines (FR-29), Background (FR-30), stress requirement (NFR-12).
Additional design patterns?	Patterns must be used; they will be asked about at the end of the term.	Required patterns are a constraint (NFR-21); further patterns are decided in the SDD.
UML notation and tool?	No restriction (draw.io, PlantUML, Mermaid).	All diagrams are written in PlantUML; sources are in the repository.

1.3 Objectives and success criteria of the project

Table 2: *Objectives and measurable success criteria*

ID	Objective	Success criterion
O-1	Automate scenario writing (P1)	A file with 100 valid EARS requirements is converted into .feature files with one command in less than 5 seconds.
O-2	Correct mapping (P2)	For a test set that covers all five EARS patterns and their combinations, 100% of the generated steps use the keyword defined by the mapping rules of Section 3.1.

ID	Objective	Success criterion
O-3	Consistent output (P2)	Two runs on the same input with the same options (LLM disabled) produce byte-identical output files.
O-4	Valid Gherkin (P2)	Every generated .feature file is accepted by the official Cucumber Gherkin parser without errors.
O-5	Clear error feedback (P2)	Every invalid requirement of the input appears in the error report with its line number, reason and expected pattern.
O-6	Traceability (P3)	Every generated scenario carries a tag with the ID of the requirement it was derived from.
O-7	Edge-case coverage (P4)	With the options enabled, every event-driven, unwanted-behaviour and optional-feature requirement of the test set gets a negative scenario, and every requirement with a numeric limit gets a boundary Scenario Outline.
O-8	Free-text support	With LLM normalization enabled, at least 16 of a sample of 20 free-text requirements are normalized into valid EARS sentences that the Test Engineer accepts.
O-9	Extensibility	A new input or output format can be added without changing the existing generation rules (checked in the design review).

1.4 Definitions, acronyms, and abbreviations

Term	Meaning
AST	Abstract Syntax Tree - the common internal model of a parsed requirement
BDD	Behavior-Driven Development
CI	Continuous Integration
CLI / GUI	Command-Line Interface / Graphical User Interface
EARS	Easy Approach to Requirements Syntax [4]
FR / NFR	Functional Requirement / Non-functional Requirement
LLM	Large Language Model
MoSCoW	Must, Should, Could, Won't - a prioritisation scheme
RAD / SDD	Requirements Analysis Document / System Design Document
UC	Use Case
UML	Unified Modeling Language [10]

Domain terms such as Feature, Scenario, Step, Background and Clause are defined in Section 4, Glossary of Terms.

1.5 References

- [1] CSE 443 Project Deliverable #1 - Requirements Analysis Document, assignment description and RAD template, Gebze Technical University, 2026.
- [2] K. İnce, "Automated Gherkin and BDD Test Scenario Generator", CSE 443 term project description (problem statement), Gebze Technical University, 2026.
- [3] Team CURATORES, project introduction presentation and meeting notes with the instructor, 12 October 2026.

- [4] A. Mavin, P. Wilkinson, A. Harwood, M. Novak, "Easy Approach to Requirements Syntax (EARS)", Proc. 17th IEEE International Requirements Engineering Conference (RE'09), 2009.
- [5] Cucumber, "Gherkin Reference", https://cucumber.io/docs/gherkin/reference/
- [6] D. North, "Introducing BDD", Better Software Magazine, 2006.
- [7] M. Cohn, User Stories Applied: For Agile Software Development, Addison-Wesley, 2004.
- [8] B. Bruegge, A. H. Dutoit, Object-Oriented Software Engineering Using UML, Patterns, and Java, 3rd ed., Prentice Hall, 2010.
- [9] R. J. Abbott, "Program Design by Informal English Descriptions", Communications of the ACM, 26(11), 1983.
- [10] Object Management Group, Unified Modeling Language Specification, Version 2.5.1, 2017.

1.6 Overview

Section 2 describes how test scenarios are written today and the problems of this practice. Section 3 presents the proposed system: an overview with the mapping rules, the functional requirements in EARS form, the non-functional requirements, and the system models - scenarios, use case model, object model, dynamic model and user interface mock-ups. Section 4 is the glossary, Section 5 gives the priority of every requirement and use case, and Section 6 traces the problem statement and the problems P1-P4 to objectives, use cases, requirements and domain classes.

2 Current system

There is no existing system to be replaced; this is a greenfield project [8]. Today the tasks supported by the new system are done manually:

1. A business analyst writes the requirements in a document or an issue tracker - sometimes in EARS form, sometimes as user stories, often as free text.
2. A test engineer reads each requirement and writes one or more Gherkin scenarios by hand in a text editor or IDE.
3. Developers implement the step definitions, and Cucumber (or a similar tool) runs the scenarios.
4. When a requirement changes, someone must remember to find and update the related scenarios.

The following as-is scenario shows this practice with concrete people and data.

Scenario name	writeScenariosByHand (as-is scenario)
Participating actors	ali: Test Engineer, deniz: Business Analyst
Flow of events	1. Deniz sends Ali 40 new requirements for the payment module in a Word file. 25 are EARS sentences, 10 are user stories and 5 are free text. 2. Ali opens payment.feature in his IDE and writes one scenario per requirement. For each sentence he decides himself which part is a Given, a When or a Then. 3. After three hours he has 41 scenarios. For "If the card is expired, then the system shall reject the payment" he writes only the rejection case; the case of a valid card is not written. 4. One week later Deniz changes the limit in REQ-12 from 3 to 5 failed attempts. Ali does not see the change, and the scenario still tests the old limit. 5. In the sprint review nobody can tell which requirements are covered by a scenario, because the scenarios do not refer to requirement IDs.

Table 3: *Problems of the current practice*

ID	Problem	Effect
P1	Writing scenarios by hand is slow.	The test effort grows linearly with the number of requirements; scenario writing becomes a bottleneck (step 2-3 above).
P2	Different people translate the same requirement in different ways.	Inconsistent step wording, missing preconditions, duplicated or conflicting scenarios.
P3	There is no link between a requirement and its scenarios.	Changed requirements leave outdated tests; coverage cannot be measured (step 4-5).
P4	Negative and edge cases are left out.	Defects at limits and in error paths are found late (step 3).

Existing tools such as Cucumber execute scenarios but do not create them. Test management plug-ins store Gherkin scenarios, but the scenarios are still written by hand. General-purpose AI assistants can draft scenarios, but their output is not deterministic and does not follow fixed mapping rules, so P2 and P3 remain.

3 Proposed system

This section documents the requirements elicitation and the analysis model of the new system.

3.1 Overview

The Test Engineer gives the system a requirements file. The system detects or uses the selected input format, validates each requirement, maps the clauses of each valid requirement to Gherkin steps with fixed rules, groups the scenarios into features, shows a preview with a summary, and writes the output files in the selected format. Invalid requirements are collected in an error report. Free-text requirements can be sent to an external LLM Service, which rewrites them as EARS sentences; the Test Engineer accepts or rejects each rewrite before it is used.

Table 4: *Mapping of EARS patterns to Gherkin steps*

EARS pattern	Structure [4]	Gherkin mapping
Ubiquitous	The <system> shall <response>.	Given the system is running → Then <response>
Event-driven	When <trigger>, the <system> shall <response>.	When <trigger> → Then <response>
State-driven	While <state>, the <system> shall <response>.	Given <state> → Then <response>
Unwanted behaviour	If <condition>, then the <system> shall <response>.	When <condition> → Then <response>
Optional feature	Where <feature>, the <system> shall <response>.	Given <feature> → Then <response>
Combination	While <state>, when <trigger>, the <system> shall <response>.	Given <state> → When <trigger> → Then <response>

In short: **While / Where → Given, When / If → When, shall → Then.** A user story becomes a Feature, and each of its acceptance criteria becomes a Scenario. On request, the system adds a negative scenario (the trigger or condition is not met) and a boundary Scenario Outline (values below, at and above a numeric limit). Given steps shared by all scenarios of a feature are moved into a Background, so that the set-up is written only once. The example below shows the four kinds of output for two requirements of the same feature.

Example - EARS input and generated Gherkin (negative and boundary options enabled)

```
Input : Feature: Account security
        REQ-7: While the account is active, when the user enters a wrong password, the
               system shall display an error message.
        REQ-9: While the account is active, if the user enters a wrong password three times,
               then the system shall lock the account.

Output: Feature: Account security

          Background:
            Given the account is active

          @REQ-7 @positive
          Scenario: The user enters a wrong password
            When the user enters a wrong password
            Then the system displays an error message

          @REQ-7 @negative
          Scenario: The user enters a wrong password - negative
            When the condition "the user enters a wrong password" is not met
            Then the system does not display an error message

          @REQ-9 @positive
          Scenario: The user enters a wrong password three times
            When the user enters a wrong password three times
            Then the system locks the account

          @REQ-9 @negative
          Scenario: The user enters a wrong password three times - negative
            When the condition "the user enters a wrong password three times" is not met
            Then the system does not lock the account

          @REQ-9 @boundary
          Scenario Outline: The user enters a wrong password <count> times
            When the user enters a wrong password <count> times
            Then the system <outcome>
            Examples:
              | count | outcome                      |
              | 2     | does not lock the account    |
              | 3     | locks the account            |
              | 4     | locks the account            |
```

3.2 Functional requirements

Functional requirements are grouped by use case. Each requirement has a unique ID, is written with an EARS pattern [4] and has a priority (Section 5). The column "EARS type" names the pattern used. "Test Engineer" is the only human actor; "LLM Service" is an external system actor. In total there are 59 functional requirements.

UC-1 Load Requirements File

ID	Requirement	EARS type	Priority
FR-1	The system shall allow the Test Engineer to load requirements from a UTF-8 plain-text file with the extension .txt or .md.	Ubiquitous	Must
FR-2	If the selected requirements file does not exist or cannot be read, then the system shall display an error message that names the file and stop the generation session.	Unwanted behaviour	Must

ID	Requirement	EARS type	Priority
FR-3	The system shall treat each non-empty line of the requirements file that does not start with "#" or "Feature:" as one requirement, except the acceptance-criterion lines described in FR-7.	Ubiquitous	Must
FR-4	Where a requirement line starts with an identifier followed by a colon (e.g., "REQ-7:"), the system shall store that identifier as the requirement ID.	Optional feature	Should
FR-5	If a requirement has no identifier, then the system shall assign the ID "L<line number>" to it.	Unwanted behaviour	Should
FR-6	Where the requirements file contains a line starting with "Feature:", the system shall assign the following requirements to a feature with that name until the next "Feature:" line.	Optional feature	Should
FR-7	When a line that starts with "Given", "When", "Then" or "And", does not contain the word "shall" and directly follows a user story or another such line, the system shall store the line as a step of an acceptance criterion of that user story.	Event-driven	Should
FR-8	When loading completes, the system shall display the number of requirements read.	Event-driven	Must

UC-2 Select Input Format

ID	Requirement	EARS type	Priority
FR-9	The system shall support three input formats: EARS requirement, user story and free text.	Ubiquitous	Must
FR-10	When the Test Engineer selects an input format, the system shall process every requirement in the loaded file according to that format.	Event-driven	Must
FR-11	If no input format is selected, then the system shall detect the format of each requirement from its leading keyword ("When", "While", "If", "Where" or "As a") or, when the requirement starts with "The", from the word "shall" in it.	Unwanted behaviour	Should
FR-12	If no input format is selected and a requirement is not detected as an EARS requirement or a user story by the rules of FR-11, then the system shall classify the requirement as free text.	Unwanted behaviour	Should

UC-3 Generate Scenarios

ID	Requirement	EARS type	Priority
FR-13	The system shall recognise the five EARS patterns (ubiquitous, event-driven, state-driven, unwanted behaviour and optional feature) and their combinations in one sentence.	Ubiquitous	Must
FR-14	The system shall map each "While" and "Where" clause of a valid EARS requirement to a Given step.	Ubiquitous	Must
FR-15	The system shall map each "When" and "If" clause of a valid EARS requirement to a When step.	Ubiquitous	Must
FR-16	The system shall map each "shall" response clause of a valid EARS requirement to a Then step.	Ubiquitous	Must

ID	Requirement	EARS type	Priority
FR-17	When a Scenario contains more than one step with the same keyword in sequence, the system shall write the second and later steps with the keyword "And".	Event-driven	Must
FR-18	When a ubiquitous requirement is translated, the system shall add the step "Given the system is running" before its Then step.	Event-driven	Must
FR-19	The system shall derive the Scenario title from the trigger or condition clause of the requirement, or from the response clause when the requirement has no trigger or condition.	Ubiquitous	Must
FR-20	The system shall translate each valid user story into one Feature whose title is derived from the goal of the story and whose description contains the role, goal and benefit of the story.	Ubiquitous	Should
FR-21	Where a user story has acceptance criteria, the system shall create one Scenario for each acceptance criterion.	Optional feature	Should
FR-22	If a valid user story has no acceptance criteria, then the system shall create one Scenario whose When step is derived from the goal and whose Then step is derived from the benefit.	Unwanted behaviour	Could
FR-23	The system shall tag every generated Scenario with the ID of the requirement it was derived from (e.g., "@REQ-7").	Ubiquitous	Must
FR-24	The system shall tag every generated Scenario with its kind: "@positive", "@negative" or "@boundary".	Ubiquitous	Should
FR-25	The system shall group the generated Scenarios into Features according to the feature names of FR-6.	Ubiquitous	Must
FR-26	The system shall group the Scenarios of requirements without a feature name into one Feature named after the input file.	Ubiquitous	Must
FR-27	If two Scenarios in the same Feature have the same title, then the system shall append a sequence number to the title of the second Scenario.	Unwanted behaviour	Should
FR-28	Where negative scenario generation is enabled, the system shall create, for each event-driven, unwanted-behaviour or optional-feature requirement, one negative Scenario in which the trigger or condition is not met and the Then step states that the response does not occur.	Optional feature	Should
FR-29	Where boundary scenario generation is enabled and a clause of a requirement contains a numeric limit (e.g., "three times", "more than 5 items"), the system shall create one Scenario Outline whose Examples table contains the values just below, at and just above the limit.	Optional feature	Could
FR-30	When all Scenarios of a Feature start with the same Given steps, the system shall move these steps into a Background section of that Feature.	Event-driven	Should

UC-4 Validate Requirements

ID	Requirement	EARS type	Priority
FR-31	When generation starts, the system shall validate every loaded requirement before it generates any scenario.	Event-driven	Must
FR-32	If an EARS requirement does not contain a "shall" response clause, then the system shall mark the requirement as invalid with the reason "missing response clause".	Unwanted behaviour	Must

ID	Requirement	EARS type	Priority
FR-33	If a requirement does not match any of the five EARS patterns, a combination of them or the user story template, then the system shall mark the requirement as invalid and state the closest expected pattern.	Unwanted behaviour	Must
FR-34	If a user story does not contain both a role ("As a") and a goal ("I want"), then the system shall mark the requirement as invalid with the reason "incomplete user story".	Unwanted behaviour	Should
FR-35	If two requirements have the same ID, then the system shall mark the second requirement as invalid with the reason "duplicate requirement ID".	Unwanted behaviour	Should
FR-36	If an EARS requirement contains more than one "shall" response clause, then the system shall record the warning "compound requirement" and create one Then step for each response clause.	Unwanted behaviour	Could
FR-37	While at least one requirement is valid, the system shall continue generation for the valid requirements even if other requirements are invalid.	State-driven	Must

UC-5 View Error Report

ID	Requirement	EARS type	Priority
FR-38	When validation finds at least one invalid requirement, the system shall create an error report.	Event-driven	Must
FR-39	The system shall include the line number, requirement ID, requirement text, reason, expected pattern and a correction hint in each error report entry.	Ubiquitous	Must
FR-40	When the Test Engineer requests the error report, the system shall display it with the entries ordered by line number.	Event-driven	Must
FR-41	Where the Test Engineer gives a report file path, the system shall also save the error report to that file.	Optional feature	Could

UC-6 Preview Scenarios

ID	Requirement	EARS type	Priority
FR-42	When generation completes, the system shall display a preview of all generated Features, Backgrounds and Scenarios in Gherkin form.	Event-driven	Must
FR-43	When generation completes, the system shall display a summary with the number of requirements read, valid, invalid and normalized, and the number of generated Scenarios.	Event-driven	Must
FR-44	Where the option "--preview-only" is given, the system shall not write any output file.	Optional feature	Should

UC-7 Export Scenarios

ID	Requirement	EARS type	Priority
FR-45	When the Test Engineer confirms the export, the system shall write one output file per Feature into the selected output directory.	Event-driven	Must
FR-46	The system shall name each output file after its Feature title in lower case, with spaces replaced by underscores, and with the extension of the output format.	Ubiquitous	Must

ID	Requirement	EARS type	Priority
FR-47	If the selected output directory does not exist, then the system shall create it.	Unwanted behaviour	Should
FR-48	If an output file with the same name already exists, then the system shall ask the Test Engineer for confirmation before overwriting it.	Unwanted behaviour	Should
FR-49	If an output file cannot be written, then the system shall report the file path and the cause and keep the files already written unchanged.	Unwanted behaviour	Should
FR-50	When the export completes, the system shall list the paths of all written files.	Event-driven	Must

UC-8 Select Output Format

ID	Requirement	EARS type	Priority
FR-51	The system shall support three output formats: Gherkin (.feature), Markdown (.md) and JSON (.json).	Ubiquitous	Must
FR-52	If no output format is selected, then the system shall use the Gherkin format.	Unwanted behaviour	Must

UC-9 Normalize Free-Text Requirement

ID	Requirement	EARS type	Priority
FR-53	Where LLM normalization is enabled, the system shall send each free-text requirement to the configured LLM Service and request an equivalent requirement in EARS form.	Optional feature	Could
FR-54	If the text returned by the LLM Service does not pass the validation rules of FR-32 and FR-33, then the system shall mark the free-text requirement as invalid with the reason "normalization failed".	Unwanted behaviour	Could
FR-55	When a normalized requirement passes validation, the system shall display the original text and the normalized text and ask the Test Engineer to accept or reject the normalized text.	Event-driven	Could
FR-56	If the Test Engineer rejects a normalized text, then the system shall mark the requirement as invalid with the reason "normalization rejected".	Unwanted behaviour	Could
FR-57	The system shall tag every Scenario derived from a normalized requirement with "@llm-normalized".	Ubiquitous	Could
FR-58	If LLM normalization is not enabled, then the system shall mark each free-text requirement as invalid with the reason "free text requires LLM normalization".	Unwanted behaviour	Should
FR-59	If the LLM Service does not answer within 30 seconds or returns an error, then the system shall mark the free-text requirement as invalid with the reason "LLM service unavailable" and continue with the next requirement.	Unwanted behaviour	Could

3.3 Nonfunctional requirements

Non-functional requirements are numbered NFR-1 … NFR-33 and are also written with EARS patterns. They are measurable wherever possible, as required in the lecture on requirements elicitation. Implementation, interface, packaging and legal requirements are constraints ("pseudo requirements") [8].

3.3.1 Usability

ID	Requirement	EARS type	Priority
NFR-1	The system shall display a help text that lists all commands and options when it is started with the option "--help".	Ubiquitous	Should
NFR-2	The system shall state, in every validation error message, the line number, the reason and the expected pattern.	Ubiquitous	Must
NFR-3	The system shall allow a user who knows EARS to generate a first .feature file within 10 minutes using only the README.	Ubiquitous	Should
NFR-4	The system shall keep the wording of the requirement in the generated steps and change only the verb form of the response clause (e.g., "shall display" becomes "displays").	Ubiquitous	Should
NFR-5	Where the graphical interface is used, the system shall allow the Test Engineer to go from loading a file to exporting the scenarios with at most four user actions.	Optional feature	Could

3.3.2 Reliability

ID	Requirement	EARS type	Priority
NFR-6	If the input contains a malformed requirement, then the system shall report the requirement and continue without abnormal termination.	Unwanted behaviour	Must
NFR-7	While LLM normalization is disabled, the system shall produce byte-identical output for identical input and options.	State-driven	Must
NFR-8	If writing an output file fails, then the system shall not leave a partially written file in the output directory.	Unwanted behaviour	Should
NFR-9	The system shall distinguish errors in the user input from internal errors of the system in every error message (prefix "INPUT ERROR" or "INTERNAL ERROR").	Ubiquitous	Should

3.3.3 Performance

ID	Requirement	EARS type	Priority
NFR-10	While LLM normalization is disabled, the system shall process an input file of 500 requirements and write the output in less than 5 seconds on a standard laptop (4 cores, 8 GB RAM).	State-driven	Should
NFR-11	The system shall start and display the help text in less than 2 seconds.	Ubiquitous	Could
NFR-12	The system shall process an input file of 10,000 requirements without abnormal termination and with less than 512 MB of memory.	Ubiquitous	Should

3.3.4 Supportability

ID	Requirement	EARS type	Priority
NFR-13	The system shall allow a new input format to be added without changes to the existing validation, generation or output rules.	Ubiquitous	Should
NFR-14	The system shall allow a new output format to be added without changes to the existing input or generation rules.	Ubiquitous	Should
NFR-15	The system shall allow a new validation rule to be added without changes to the existing validation rules.	Ubiquitous	Should

ID	Requirement	EARS type	Priority
NFR-16	The system shall allow the LLM Service provider to be changed through the configuration file without changes to the source code.	Ubiquitous	Could
NFR-17	The system shall have automated unit tests that cover at least 70% of the source lines.	Ubiquitous	Should
NFR-18	Where the option "--verbose" is given, the system shall log each processing step to the console.	Optional feature	Could

3.3.5 Implementation

ID	Requirement	EARS type	Priority
NFR-19	The system shall be implemented in Java 17 or later and built with Maven or Gradle.	Ubiquitous	Must
NFR-20	The system shall translate EARS requirements and user stories with rule-based parsing and use the LLM Service only for the optional normalization of free text.	Ubiquitous	Must
NFR-21	The design of the system shall apply the Builder, Adapter and Chain of Responsibility patterns, as required by the problem statement [2].	Ubiquitous	Must
NFR-22	The source code and all documents shall be kept in the team's Git repository on GitHub.	Ubiquitous	Should

3.3.6 Interface

ID	Requirement	EARS type	Priority
NFR-23	The system shall read UTF-8 encoded text files as input.	Ubiquitous	Must
NFR-24	The system shall produce .feature files that follow the Cucumber Gherkin syntax [5].	Ubiquitous	Must
NFR-25	The system shall return exit code 0 on success, 1 when some requirements are invalid, and 2 on a fatal error, so that it can be used in CI pipelines.	Ubiquitous	Should
NFR-26	The system shall document the structure of its JSON output with a JSON Schema delivered with the system.	Ubiquitous	Could
NFR-27	The system shall read the API key of the LLM Service only from an environment variable and never write the key to an output file or log.	Ubiquitous	Could
NFR-28	Where the graphical interface is installed, the system shall offer the functions of UC-1 to UC-9 in one desktop window.	Optional feature	Could

3.3.7 Packaging

ID	Requirement	EARS type	Priority
NFR-29	The system shall be delivered as a single executable JAR file together with a README and sample input files.	Ubiquitous	Must
NFR-30	The system shall start the graphical interface from the same JAR file with the command "gherkingen gui".	Ubiquitous	Could

3.3.8 Legal

ID	Requirement	EARS type	Priority
NFR-31	The system shall use only third-party libraries with permissive open-source licenses (e.g., MIT, Apache 2.0).	Ubiquitous	Should
NFR-32	While LLM normalization is disabled, the system shall not send any input data over the network.	State-driven	Must
NFR-33	When LLM normalization is enabled for the first time, the system shall warn the Test Engineer that free-text requirements will be sent to a third-party service and ask for confirmation.	Event-driven	Could

3.4 System models

This section contains the scenarios, the use case model, the object model, the dynamic model and the user interface mock-ups of the system. All diagrams use UML 2.5 notation [10] and were written in PlantUML; their sources are kept in the folder docs/diagrams of the repository.

3.4.1 Scenarios

The scenarios below are visionary scenarios [8]: concrete, informal descriptions of one use of the future system by one actor instance. Each scenario lists the use cases it exercises.

Scenario name	generateLoginScenarios
Participating actors	ayse: Test Engineer
Flow of events	1. Ayşe receives login.txt from the business analyst. It contains six EARS requirements with the IDs REQ-1 … REQ-6 under the line "Feature: User login". 2. She runs "gherkingen generate login.txt --in ears --preview-only". 3. The system reads six requirements, validates them and finds all of them valid. 4. The system shows the preview of the feature "User login" with six scenarios and the summary "6 read | 6 valid | 0 invalid | 0 normalized | 6 scenarios". 5. Ayşe checks that "While the user is on the login page, when the user enters a wrong password, the system shall display an error message" became one Given, one When and one Then step, tagged @REQ-3. 6. She runs the command again without --preview-only and with "-o features/". The system writes features/user_login.feature and lists the path. 7. Ayşe commits the file; Cucumber parses it without errors.
Use cases	UC-1, UC-2, UC-3, UC-4, UC-6, UC-7, UC-8

Scenario name	fixInvalidRequirements
Participating actors	mehmet: Test Engineer
Flow of events	1. Mehmet loads payment.txt, which contains ten requirements. 2. The system validates them and finds two invalid requirements: line 4 has no "shall" clause, and line 9 uses the same ID as line 2. 3. The system generates eight scenarios and shows the summary "10 read | 8 valid | 2 invalid | 0 normalized | 8 scenarios" with a warning. 4. Mehmet opens the error report. Each entry shows the line number, ID, text, reason, expected pattern and a hint, for example "add 'the system shall …' before the response".

	5. He saves the report with "--report errors.txt" and sends it to the analyst. After the analyst corrects the two lines, Mehmet runs the generation again; all ten requirements are valid and ten scenarios are generated.
Use cases	UC-1, UC-3, UC-4, UC-5

Scenario name	userStoriesToMarkdown
Participating actors	zeynep: Test Engineer
Flow of events	1. Zeynep has cart.txt with three user stories whose goals are "add items to the cart", "remove items from the cart" and "apply a discount code". Two of them have acceptance criteria written as Given / When / Then lines. 2. She runs the generation with "--in story --out markdown -o docs/". 3. The system creates one Feature per user story and derives its title from the goal of the story. The two stories with acceptance criteria get one Scenario per criterion; the third story gets one Scenario derived from its goal and benefit. 4. The file docs/add_items_to_the_cart.md already exists from an earlier run, so the system asks "docs/add_items_to_the_cart.md already exists. Overwrite? (y/n)". Zeynep answers "y". 5. The system writes one Markdown file per Feature - docs/add_items_to_the_cart.md, docs/remove_items_from_the_cart.md and docs/apply_a_discount_code.md - and lists their paths. Zeynep shares the files with the product owner for review.
Use cases	UC-1, UC-2, UC-3, UC-6, UC-7, UC-8

Scenario name	negativeAndBoundaryScenarios
Participating actors	can: Test Engineer
Flow of events	1. Can loads security.txt with REQ-7 and REQ-9 (the example of Section 3.1). 2. He runs "gherkingen generate security.txt --negative --boundary". 3. The system creates a positive and a negative scenario for REQ-7 and REQ-9. It finds the number "three" in REQ-9 and creates a Scenario Outline with the values 2, 3 and 4. 4. All scenarios of the feature start with "Given the account is active", so the system moves this step into a Background. 5. The summary reads "2 read | 2 valid | 0 invalid | 0 normalized | 5 scenarios". Can exports the file and runs it with Cucumber in the CI pipeline; the tool returns exit code 0.
Use cases	UC-1, UC-3, UC-4, UC-6, UC-7

Scenario name	freeTextWithLlm
Participating actors	elif: Test Engineer, gpt: LLM Service
Flow of events	1. Elif has notes.txt with two free-text requirements: "Users should get an email after they reset their password" and "Make the app fast". 2. She runs the generation with "--llm". On the first use the system warns that free text will be sent to a third-party service; Elif confirms. 3. For the first line the LLM Service returns "When the user resets the password, the system shall send a confirmation email." The text is valid EARS; the system shows the original and the rewrite, and Elif accepts it.

	4. For the second line the LLM Service returns "The system shall be fast." The text is valid EARS but not testable; Elif rejects it, and the requirement is marked invalid with the reason "normalization rejected". 5. The system generates one scenario tagged @L1 and @llm-normalized and shows "2 read | 1 valid | 1 invalid | 1 normalized | 1 scenario".
Use cases	UC-1, UC-2, UC-3, UC-4, UC-5, UC-9

3.4.2 Use case model

Figure 1 shows the use case diagram. **Test Engineer** - a developer or tester who prepares acceptance tests - initiates all use cases. **LLM Service** is an external system (e.g., a hosted large language model behind an HTTPS API) that participates only in UC-9. Generate Scenarios always includes Validate Requirements. View Error Report and Normalize Free-Text Requirement extend Validate Requirements under the conditions written on the extend relations. Export Scenarios includes Select Output Format.

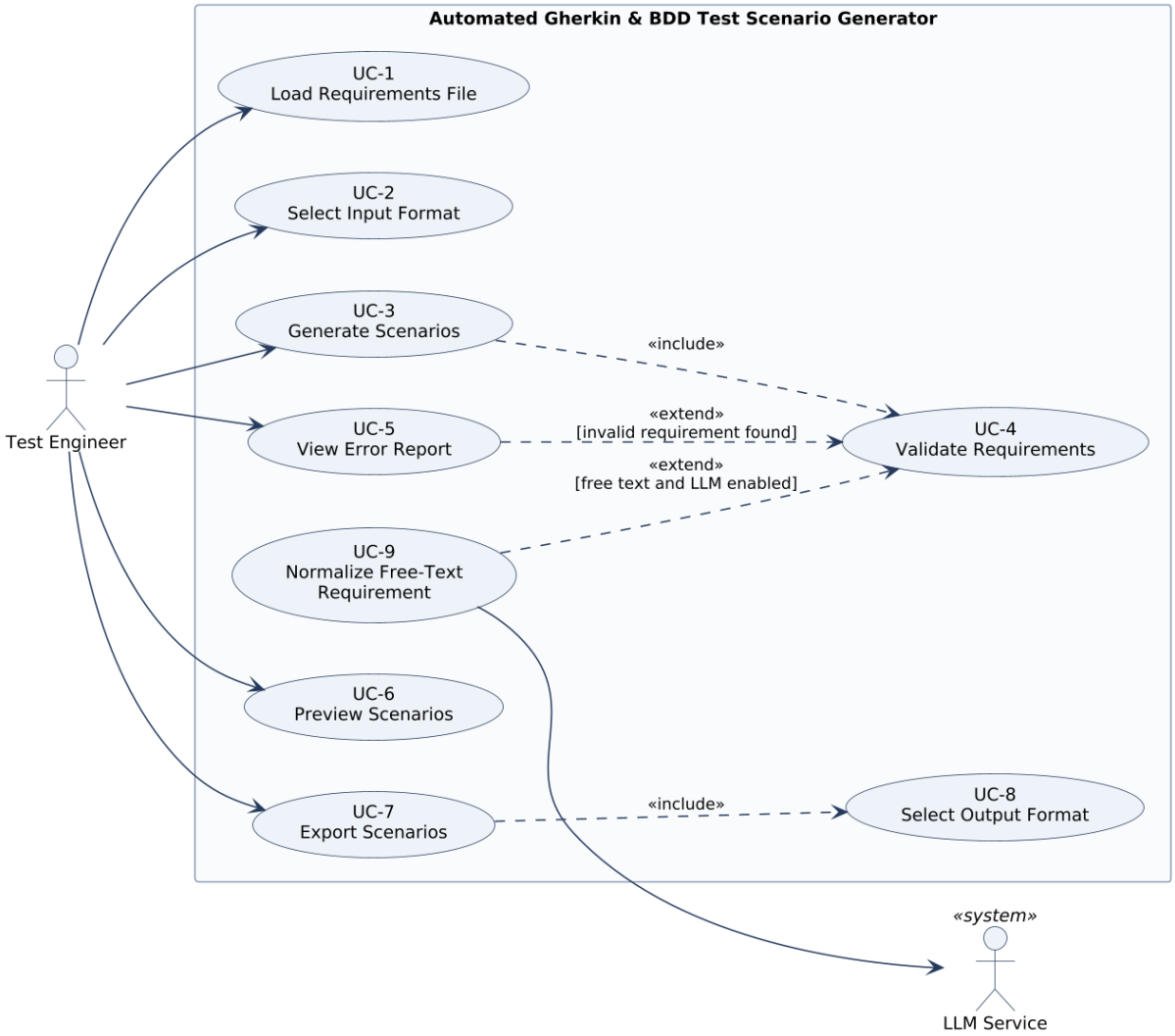


Figure 1: *Use case diagram*

UC-1 Load Requirements File

Use case	UC-1 Load Requirements File
Participating actors	Initiated by Test Engineer

Entry condition	The Test Engineer has a requirements file on the local file system.
Flow of events	1. The Test Engineer gives the path of the requirements file (CLI argument or GUI file dialog). 2. The system opens and reads the file. 3. The system creates one requirement for each non-empty, non-comment line and stores its ID, text, line number and feature name. Acceptance-criterion lines are attached to the preceding user story. 4. The system displays the number of requirements read.
Exit condition	The requirements are loaded and have the status LOADED.
Exceptions	File missing or unreadable: the system displays an error that names the file, and the use case ends (FR-2).
Related FRs	FR-1 … FR-8

UC-2 Select Input Format

Use case	UC-2 Select Input Format
Participating actors	Initiated by Test Engineer
Entry condition	The Test Engineer starts a generation session.
Flow of events	1. The Test Engineer selects "EARS", "user story" or "free text" as the input format. 2. The system uses that format for every requirement of the file.
Alternative flow	The Test Engineer selects no format: the system detects the format of each requirement from its leading keyword; a requirement without a known keyword is classified as free text.
Exit condition	Each requirement has a known input format.
Related FRs	FR-9 … FR-12

UC-3 Generate Scenarios

Use case	UC-3 Generate Scenarios
Participating actors	Initiated by Test Engineer
Entry condition	Requirements are loaded (UC-1) and their input format is known (UC-2).
Flow of events	1. The Test Engineer starts the generation and optionally enables negative and boundary scenarios. 2. The system validates all requirements (include UC-4). 3. For each valid EARS requirement, the system maps While/Where clauses to Given, When/If clauses to When and shall clauses to Then, and creates a positive Scenario. 4. For each valid user story, the system creates a Feature and one Scenario per acceptance criterion. 5. If enabled, the system adds a negative Scenario and, for numeric limits, a boundary Scenario Outline. 6. The system tags each Scenario with its requirement ID and kind, groups the Scenarios into Features and moves shared Given steps into a Background. 7. The system shows the preview and the summary (UC-6).

Exit condition	Features and Scenarios exist for all valid requirements; each valid requirement has the status TRANSLATED.
Exceptions	No valid requirement: the system shows the error report and generates nothing.
Quality requirements	Deterministic output (NFR-7); 500 requirements in less than 5 s (NFR-10).
Related FRs	FR-13 … FR-31, FR-37

UC-4 Validate Requirements

Use case	UC-4 Validate Requirements
Participating actors	Initiated by Test Engineer (through UC-3)
Entry condition	Generation has started.
Flow of events	1. The system checks each requirement against its input format and the five EARS patterns or the user story template. 2. The system checks that requirement IDs are unique. 3. The system marks each requirement VALID or INVALID and records a ValidationError with reason, expected pattern and hint for each invalid one. Compound requirements get a warning.
Extension points	invalid requirement found (UC-5); free text found while LLM normalization is enabled (UC-9)
Exit condition	Every requirement is VALID or INVALID.
Related FRs	FR-31 … FR-37

UC-5 View Error Report

Use case	UC-5 View Error Report
Participating actors	Initiated by Test Engineer
Entry condition	Extends UC-4 when at least one requirement is INVALID.
Flow of events	1. The system creates an error report from the validation errors. 2. The Test Engineer requests the report. 3. The system displays one entry per invalid requirement, ordered by line number. 4. Optionally, the Test Engineer gives a file path, and the system saves the report there.
Exit condition	The Test Engineer knows which requirements to correct and how.
Related FRs	FR-38 … FR-41

UC-6 Preview Scenarios

Use case	UC-6 Preview Scenarios
Participating actors	Initiated by Test Engineer
Entry condition	UC-3 has generated at least one Scenario.
Flow of events	1. The system displays every Feature with its Background, Scenarios and Steps in Gherkin form. 2. The system displays the summary counts.

	3. The Test Engineer reviews the preview and decides to export or to correct the input.
Exit condition	The Test Engineer has reviewed the generated scenarios.
Related FRs	FR-42 … FR-44

UC-7 Export Scenarios

Use case	UC-7 Export Scenarios
Participating actors	Initiated by Test Engineer
Entry condition	Generated Features exist and the option --preview-only is not given.
Flow of events	1. The Test Engineer gives an output directory. 2. The system determines the output format (include UC-8). 3. The system creates the directory if it does not exist. 4. The system writes one file per Feature, named after the Feature title. 5. The system lists the paths of the written files.
Alternative flow	A file with the same name exists: the system asks for confirmation; if the Test Engineer refuses, that file is skipped.
Exceptions	A file cannot be written: the system reports the path and the cause; files already written stay unchanged and no partial file remains.
Exit condition	The output files are in the output directory.
Related FRs	FR-45 … FR-50

UC-8 Select Output Format

Use case	UC-8 Select Output Format
Participating actors	Initiated by Test Engineer (through UC-7)
Entry condition	UC-7 has started.
Flow of events	1. The Test Engineer selects Gherkin, Markdown or JSON. 2. The system uses that format for all output files.
Alternative flow	No format selected: the system uses Gherkin.
Exit condition	The output format is known.
Related FRs	FR-51, FR-52

UC-9 Normalize Free-Text Requirement

Use case	UC-9 Normalize Free-Text Requirement
Participating actors	Initiated by Test Engineer (through UC-4); communicates with LLM Service
Entry condition	Extends UC-4 when a requirement is classified as free text and LLM normalization is enabled.
Flow of events	1. The system sends the free-text requirement to the LLM Service and asks for an equivalent EARS sentence. 2. The LLM Service returns a candidate sentence.

	3. The system validates the candidate with the EARS rules. 4. The system shows the original and the candidate; the Test Engineer accepts it. 5. The system creates an EARS requirement from the candidate (the free-text requirement is "normalized into" it), marks the free-text requirement NORMALIZED and tags the scenarios of the new requirement @llm-normalized.
Alternative flows	3a. The candidate is not valid EARS: the requirement is marked INVALID ("normalization failed", FR-54). 4a. The Test Engineer rejects the candidate: the requirement is marked INVALID ("normalization rejected", FR-56).
Exceptions	No answer within 30 s or a service error: the requirement is marked INVALID ("LLM service unavailable") and the system continues with the next requirement.
Exit condition	The free-text requirement is NORMALIZED or INVALID.
Quality requirements	API key never written to files or logs (NFR-27); no network use while the option is disabled (NFR-32).
Related FRs	FR-53 … FR-59

3.4.3 Object model

Identification of objects (Abbott's technique)

We found the analysis objects with Abbott's textual analysis [9] of the problem statement [2] and the flows of events of UC-1 … UC-9. Table 5 shows the mapping of parts of speech to model elements.

Table 5: *Abbott's textual analysis*

Part of speech	Example from the text	Model element
Common noun naming a role or an external system	Test Engineer, LLM Service	Actors (Test Engineer, LLM Service)
Common noun	requirements file, requirement, clause, feature, scenario, step, error report, output file	Classes RequirementsFile, Requirement, Clause, Feature, Scenario, Step, ErrorReport, OutputFile
"is a kind of"	an EARS requirement / a user story / a free-text requirement is a kind of requirement	Generalization: EarsRequirement, UserStory, FreeTextRequirement ◁ Requirement
"is a kind of"	a scenario outline is a scenario with an examples table	Generalization: ScenarioOutline ◁ Scenario
"consists of", "has"	a feature consists of scenarios; a scenario has ordered steps; a user story has acceptance criteria	Composition (Feature-Scenario, Scenario-Step, UserStory-AcceptanceCriterion)
"lists", "includes"	the error report lists the validation errors	Aggregation (ErrorReport-ValidationError)
Transitive verb	a scenario is derived from a requirement; a free-text requirement is normalized into an EARS requirement	Associations "derived from", "normalized into"
Adjective / measurable noun	line number, title, requirement ID, kind, severity	Attributes

Part of speech	Example from the text	Model element
Fixed set of values	five EARS patterns; Gherkin, Markdown or JSON; Given/When/Then/And	Enumerations EarsPattern, OutputFormat, StepKeyword
Doing verb	validate, generate, normalize, export, display	Control objects (Section "Boundary, control and entity objects"); operations are left to the design

Domain (entity) object model

Figure 2 shows the analysis-level domain model and Figure 3 the enumerations it uses. The model contains only entity classes with attributes and associations; operations, visibility and storage details are left to the design phase. Class names are the terms used in the requirements and the glossary.

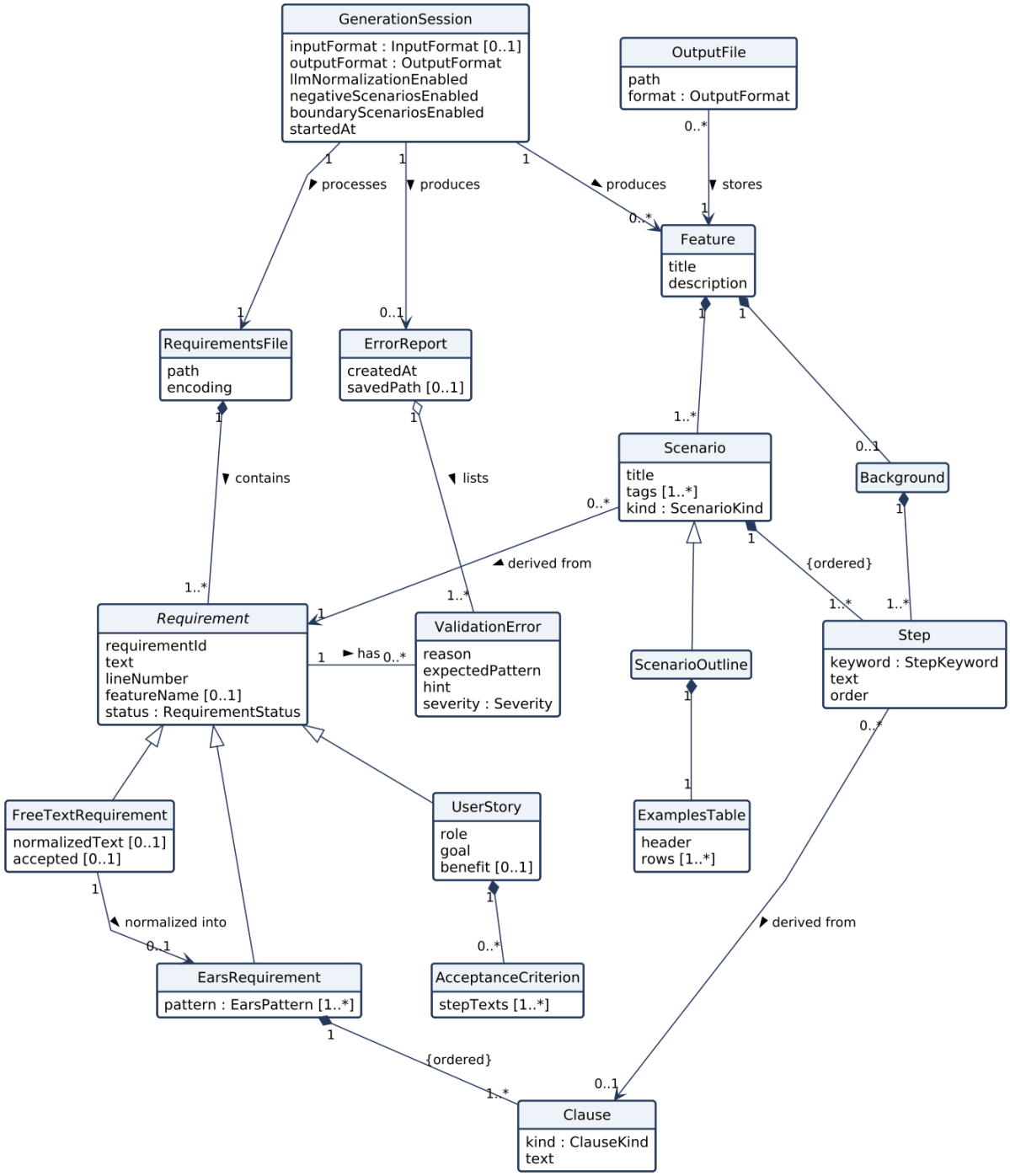


Figure 2: *Domain (entity) object model*

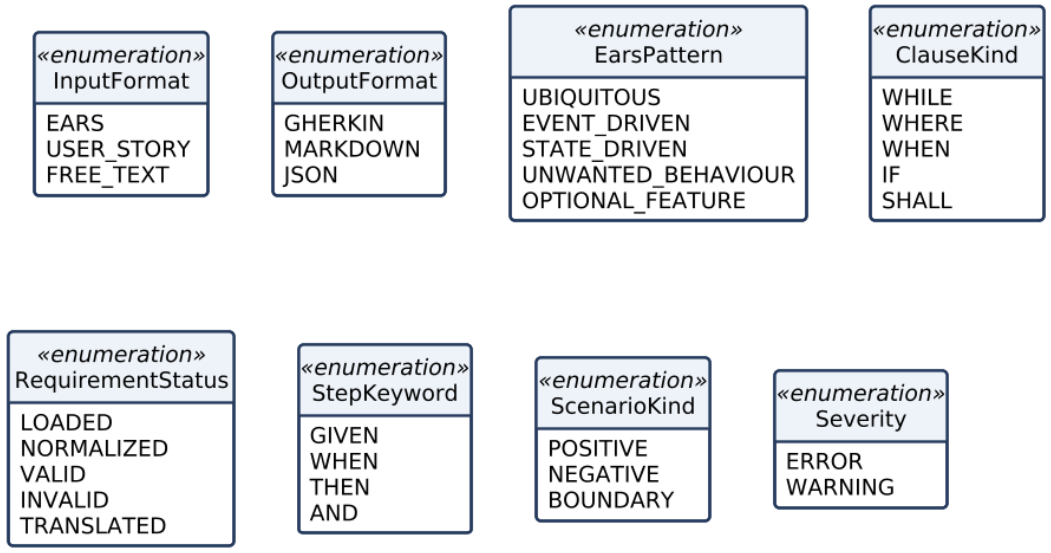


Figure 3: *Enumerations of the domain model*

Main relations:

- A **GenerationSession** processes one **RequirementsFile** with the selected formats and options and produces Features and at most one **ErrorReport**.
- A RequirementsFile contains one or more **Requirements**. Requirement is abstract; its kinds are **EarsRequirement** (an ordered list of **Clauses**), **UserStory** (role, goal, benefit and zero or more **AcceptanceCriteria**) and **FreeTextRequirement**, which can be normalized into one EarsRequirement.
- Each Requirement may have **ValidationErrors**; the ErrorReport lists them.
- A **Feature** is composed of **Scenarios** and at most one **Background**. A Scenario - or a Background - is composed of an ordered list of **Steps**. A **ScenarioOutline** is a Scenario with an **ExamplesTable**.
- Every Scenario is derived from exactly one Requirement, and a Step may be derived from one Clause. These two links realise traceability (FR-23).
- An **OutputFile** stores one Feature in one output format.

Data dictionary

Table 6: *Data dictionary of the entity classes*

Class	Description	Main attributes
GenerationSession	One run of the system on one requirements file with one set of options.	inputFormat, outputFormat, llmNormalizationEnabled, negativeScenariosEnabled, boundaryScenariosEnabled, startedAt
RequirementsFile	The plain-text input file that contains the requirements.	path, encoding
Requirement (abstract)	One requirement read from the file; the source of one or more scenarios.	requirementId, text, lineNumber, featureName, status
EarsRequirement	A requirement written with one or more EARS patterns.	pattern [1..*]
Clause	One part of an EARS sentence that starts with While, Where, When, If or shall.	kind, text

Class	Description	Main attributes
UserStory	A requirement in the form "As a <role>, I want <goal>, so that <benefit>".	role, goal, benefit
AcceptanceCriterion	A condition of a user story written as Given / When / Then lines.	stepTexts
FreeTextRequirement	A requirement that follows neither EARS nor the user story template.	normalizedText, accepted
ValidationError	The result of a failed check of one requirement.	reason, expectedPattern, hint, severity
ErrorReport	The list of all validation errors of a session.	createdAt, savedPath
Feature	A group of related scenarios; the top element of one output file.	title, description
Background	Given steps shared by all scenarios of a feature.	-
Scenario	One concrete example of system behaviour made of ordered steps.	title, tags, kind
ScenarioOutline	A scenario template that runs once for each row of its examples table.	-
ExamplesTable	The value table of a scenario outline.	header, rows
Step	One line of a scenario or background that starts with a Gherkin keyword.	keyword, text, order
OutputFile	A file written by the system that stores one feature.	path, format

Boundary, control and entity objects

Figure 4 classifies the analysis objects into boundary, control and entity objects [8]. **CommandLineInterface** and the optional **GraphicalInterface** receive the commands of the Test Engineer; **PreviewView** and **ErrorReportView** show results; **LlmServiceGateway** is the boundary to the external LLM Service. The control objects **GenerationControl**, **ValidationControl**, **NormalizationControl** and **ExportControl** each realise the flow of events of one or more use cases. Both interfaces use the same control objects, so the GUI adds no new functionality.

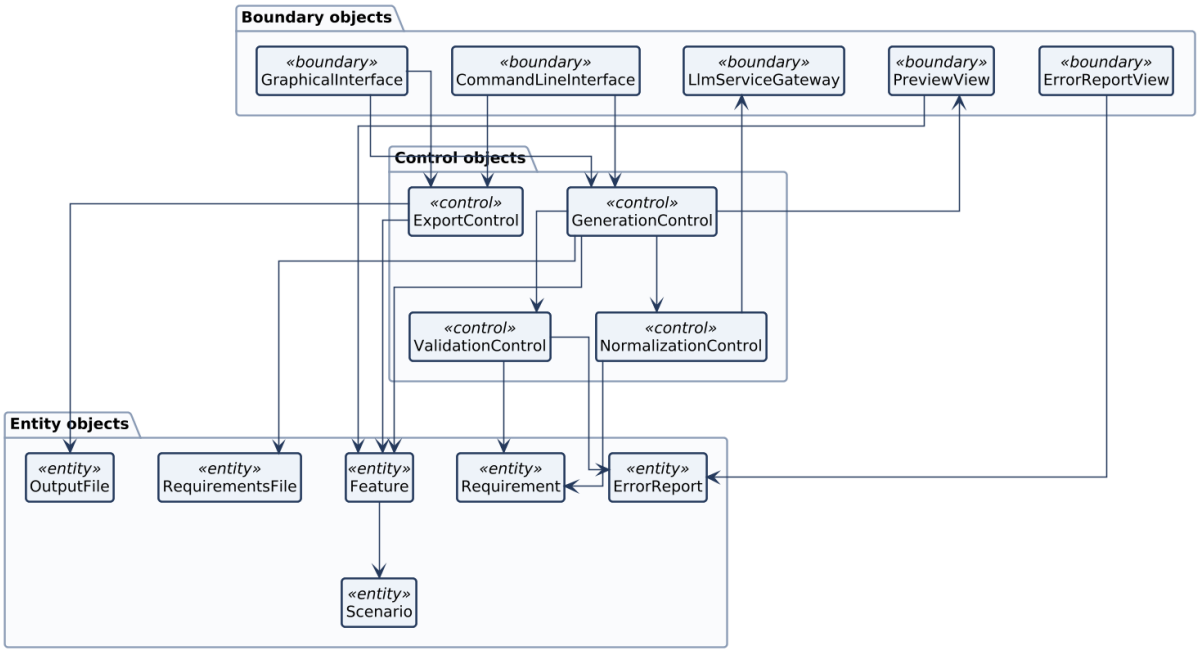


Figure 4: *Boundary, control and entity objects*

3.4.4 Dynamic model

The dynamic model describes the behaviour of the system with sequence diagrams (one per main use case), an activity diagram of a whole generation session, and state machine diagrams for the two objects with interesting behaviour [8]. Message names are informal at the analysis level; they become operations in the design.

Sequence diagrams

Figure 5 shows UC-1 Load Requirements File together with UC-2 Select Input Format.

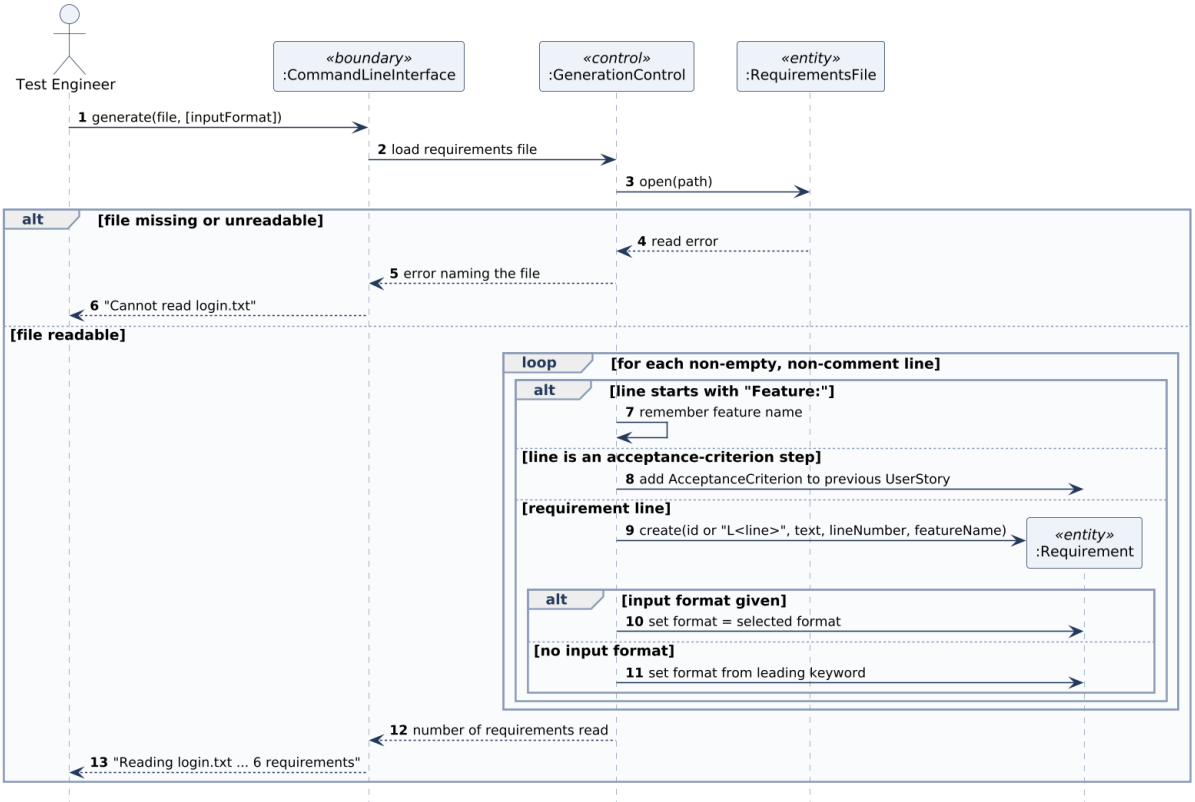


Figure 5: *Sequence diagram - Load Requirements File (UC-1, UC-2)*

Figure 6 shows UC-3 Generate Scenarios including UC-4 Validate Requirements, the optional negative and boundary scenarios and the extraction of the Background.

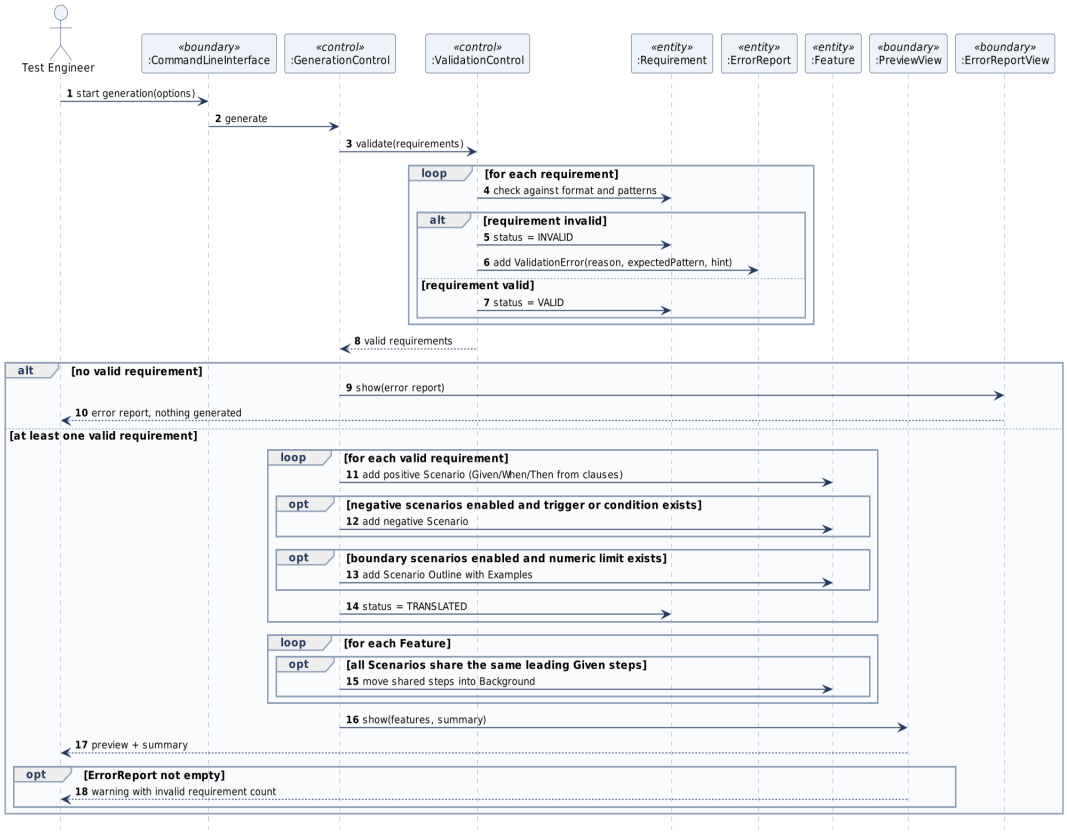


Figure 6: *Sequence diagram - Generate Scenarios (UC-3, UC-4)*

Figure 7 shows UC-9 Normalize Free-Text Requirement, which ValidationControl starts for every free-text requirement while LLM normalization is enabled.

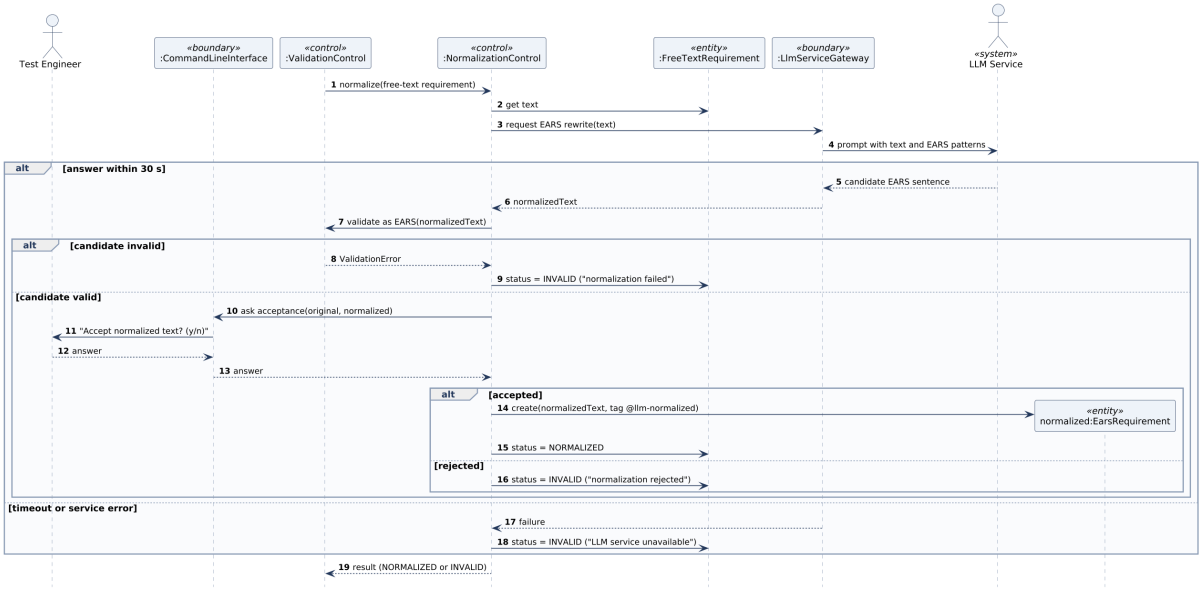


Figure 7: *Sequence diagram - Normalize Free-Text Requirement (UC-9)*

Figure 8 shows UC-5 View Error Report and Figure 9 UC-7 Export Scenarios with UC-8 and the overwrite confirmation.

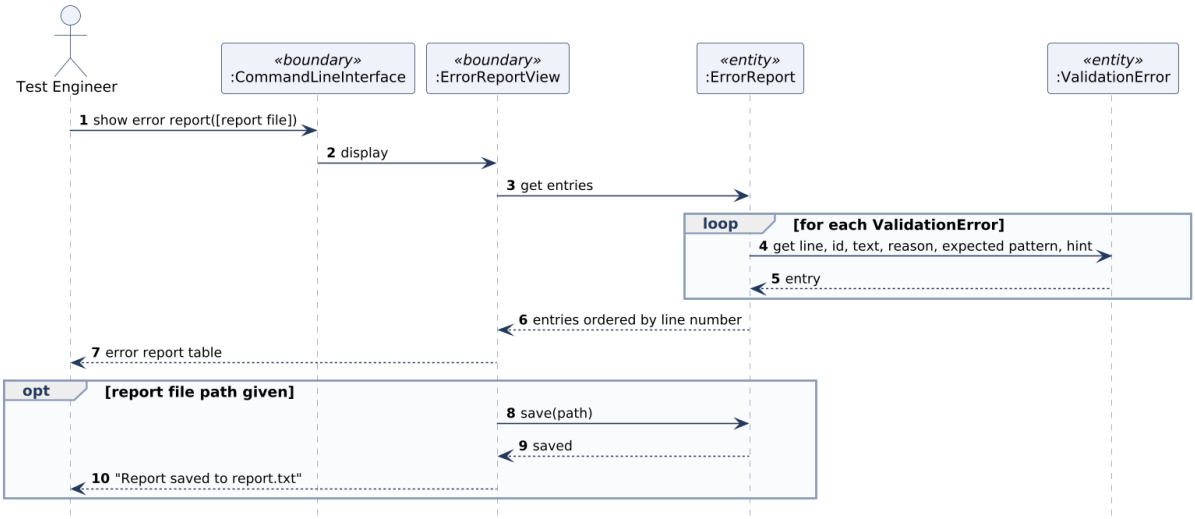


Figure 8: *Sequence diagram - View Error Report (UC-5)*

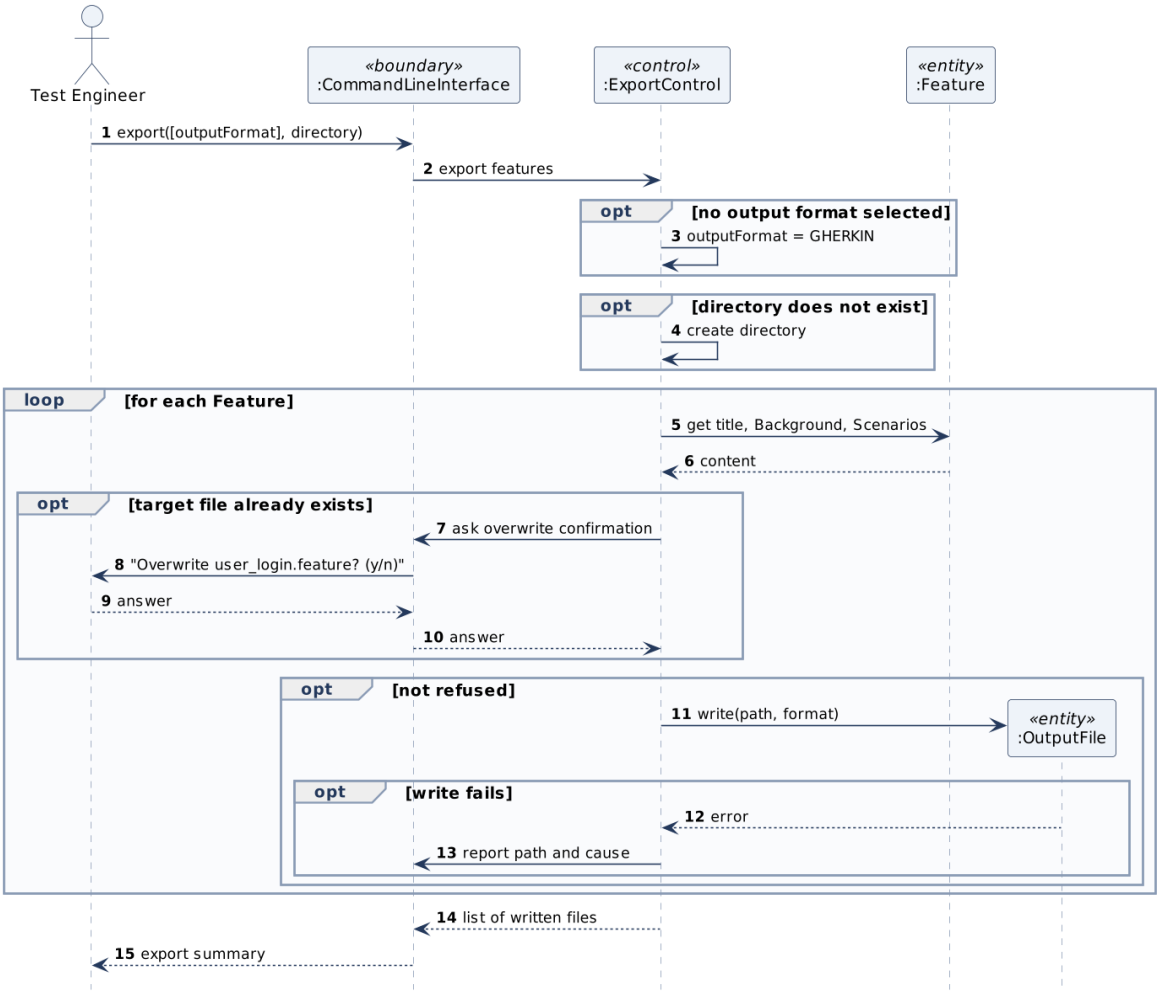


Figure 9: *Sequence diagram - Export Scenarios (UC-7, UC-8)*

Activity diagram

Figure 10 shows the overall workflow of one generation session. The three kinds of scenario are built independently of each other, which the fork shows.

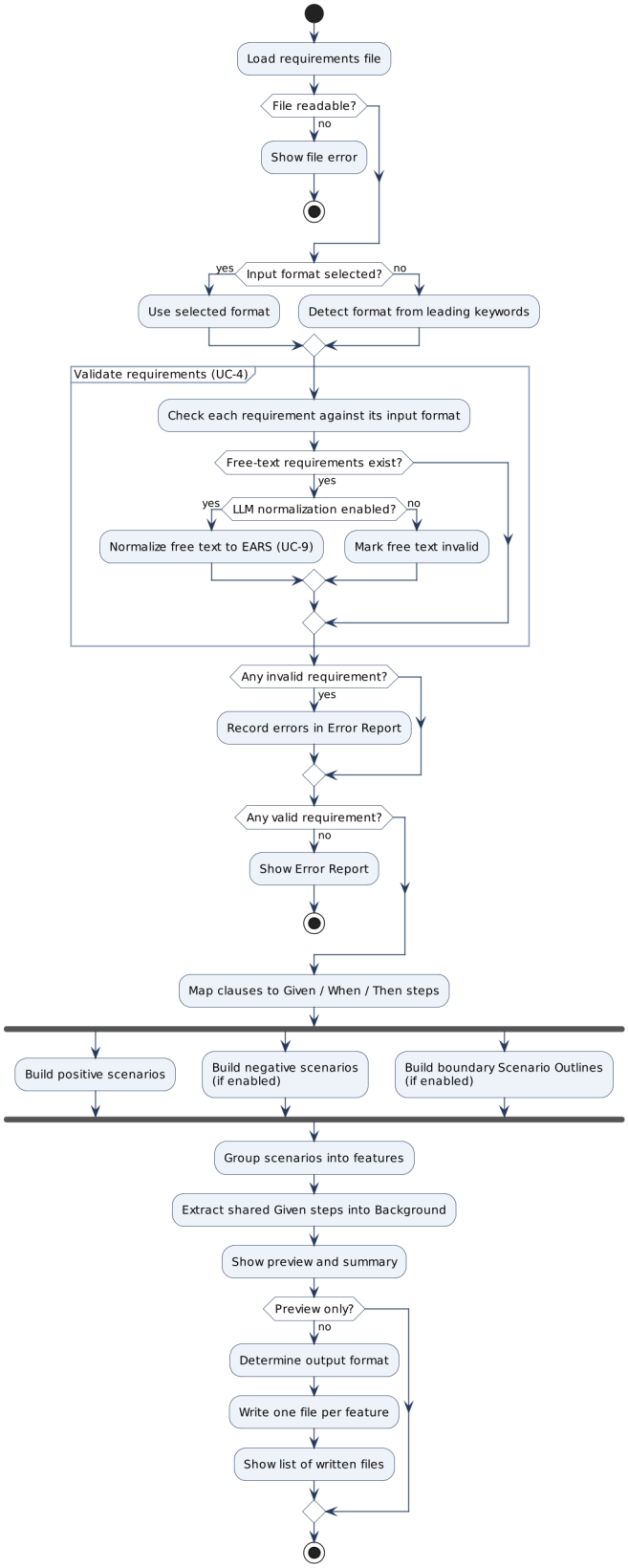


Figure 10: *Activity diagram - generation session*

State machine diagrams

Figure 11 shows the life cycle of a Requirement. The states are the values of the enumeration RequirementStatus. An invalid requirement is not repaired inside a session: the Test Engineer corrects the file and starts a new session.

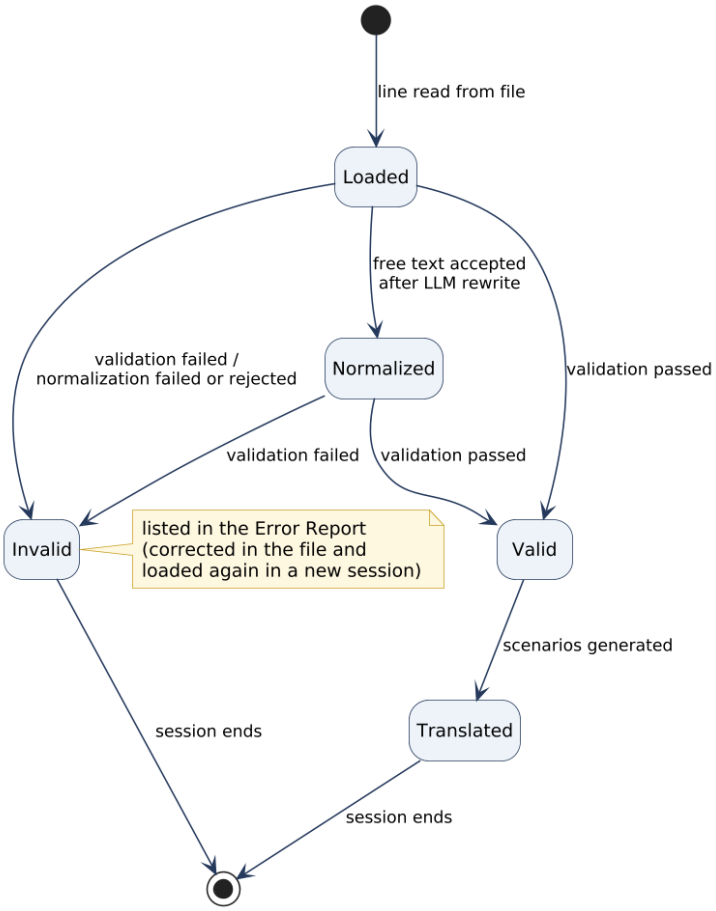


Figure 11: *State machine diagram - Requirement*

Figure 12 shows the states of a GenerationSession. Validating is a composite state, because free-text requirements are normalized while the other requirements are checked.

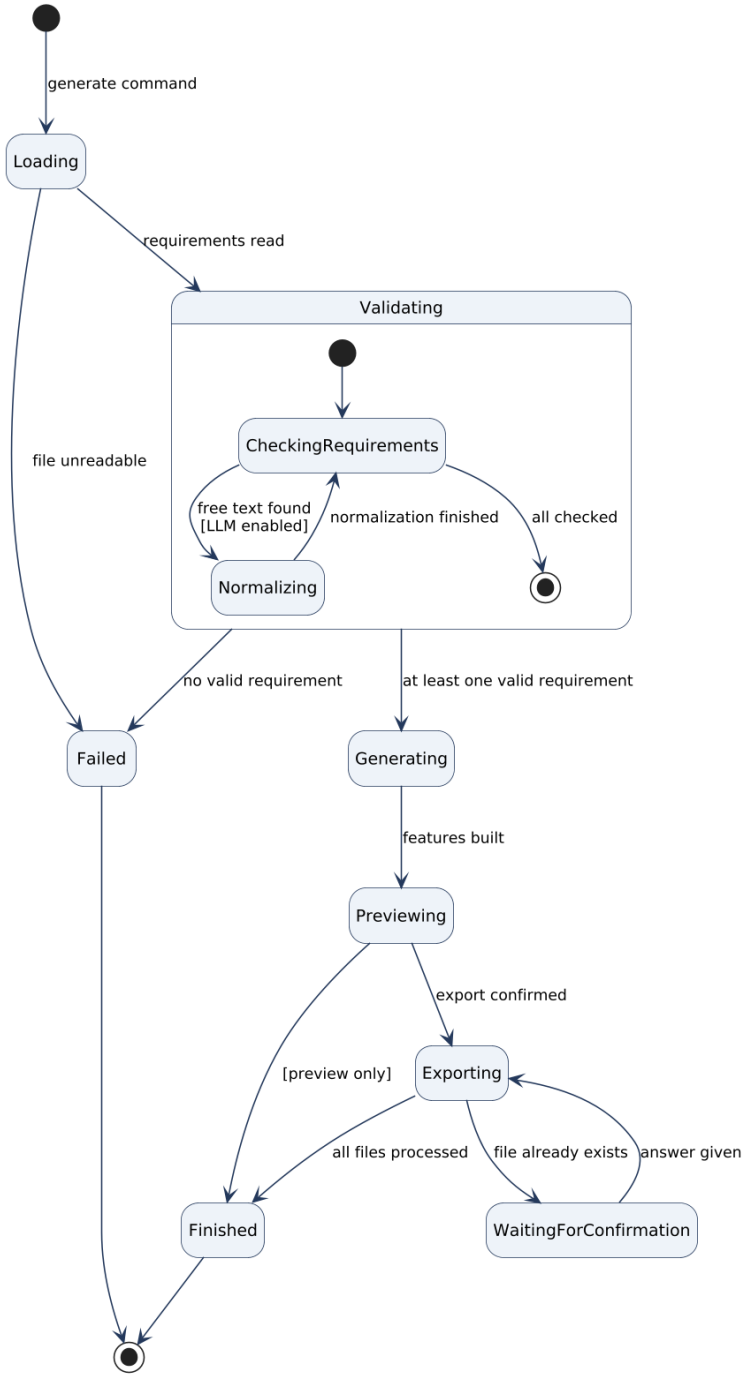


Figure 12: *State machine diagram - GenerationSession*

3.4.5 User interfaces and Mock-ups

The main interface is the command line; the command name "gherkingen" is a working name. Mock-ups 1-6 show the planned commands and screen outputs, Figure 13 the main window of the optional graphical interface, Figure 14 its navigation path between screens, and Figures 15-18 the other screens on that path.

Mock-up 1 - Help screen (NFR-1)

```
$ gherkingen --help
Usage: gherkingen generate <file> [options]
       gherkingen gui

Options:
  --in <ears|story|text>          input format (default: auto-detect)
  --out <gherkin|markdown|json>   output format (default: gherkin)
  -o, --output-dir <dir>          output directory (default: ./features)
  --negative                      also generate negative scenarios
  --boundary                      also generate boundary Scenario Outlines
  --llm                           normalize free text with the LLM Service
  --report <file>                 save the error report to a file
  --preview-only                  show the preview, do not write files
  --verbose                       log each processing step
  --help                          show this help
```

Mock-up 2 - Generation with preview and summary (UC-3, UC-6)

```
$ gherkingen generate login.txt --in ears --preview-only
Reading login.txt ... 6 requirements
Validating ......... done

Feature: User login

  @REQ-3 @positive
  Scenario: The user enters a wrong password
    Given the user is on the login page
    When the user enters a wrong password
    Then the system displays an error message
  ...

Summary: 6 read | 6 valid | 0 invalid | 0 normalized | 6 scenarios
```

Mock-up 3 - Error report (UC-5)

```
$ gherkingen generate payment.txt --report errors.txt
Summary: 10 read | 8 valid | 2 invalid | 0 normalized | 8 scenarios
WARNING: 2 requirements are invalid.

ERROR REPORT
Line  ID      Reason
4     REQ-4   INPUT ERROR: missing response clause
              "When the user clicks Pay, the payment is processed."
              expected: When <trigger>, the <system> shall <response>.
              hint: write "the system shall process the payment"
9     REQ-2   INPUT ERROR: duplicate requirement ID (first used on line 2)

Report saved to errors.txt
```

Mock-up 4 - Export with overwrite confirmation (UC-7, UC-8)

```
$ gherkingen generate cart.txt --in story --out markdown -o docs/
Summary: 3 read | 3 valid | 0 invalid | 0 normalized | 5 scenarios
docs/add_items_to_the_cart.md already exists. Overwrite? (y/n): y
Written files:
  docs/add_items_to_the_cart.md
  docs/remove_items_from_the_cart.md
  docs/apply_a_discount_code.md
```

Mock-up 5 - Negative and boundary scenarios (UC-3)

```
$ gherkingen generate security.txt --negative --boundary
Summary: 2 read | 2 valid | 0 invalid | 0 normalized | 5 scenarios
  positive: 2   negative: 2   boundary outlines: 1
  Background created for feature "Account security" (1 step)
Written files:
  features/account_security.feature
```

Mock-up 6 - Free-text normalization (UC-9)

```
$ gherkingen generate notes.txt --llm
Free-text requirements will be sent to the configured LLM Service. Continue? (y/n): y

L1  original  : Users should get an email after they reset their password
    normalized: When the user resets the password, the system shall send
                a confirmation email.
    Accept? (y/n): y
L2  original  : Make the app fast
    normalized: The system shall be fast.
    Accept? (y/n): n

Summary: 2 read | 1 valid | 1 invalid | 1 normalized | 1 scenario
```

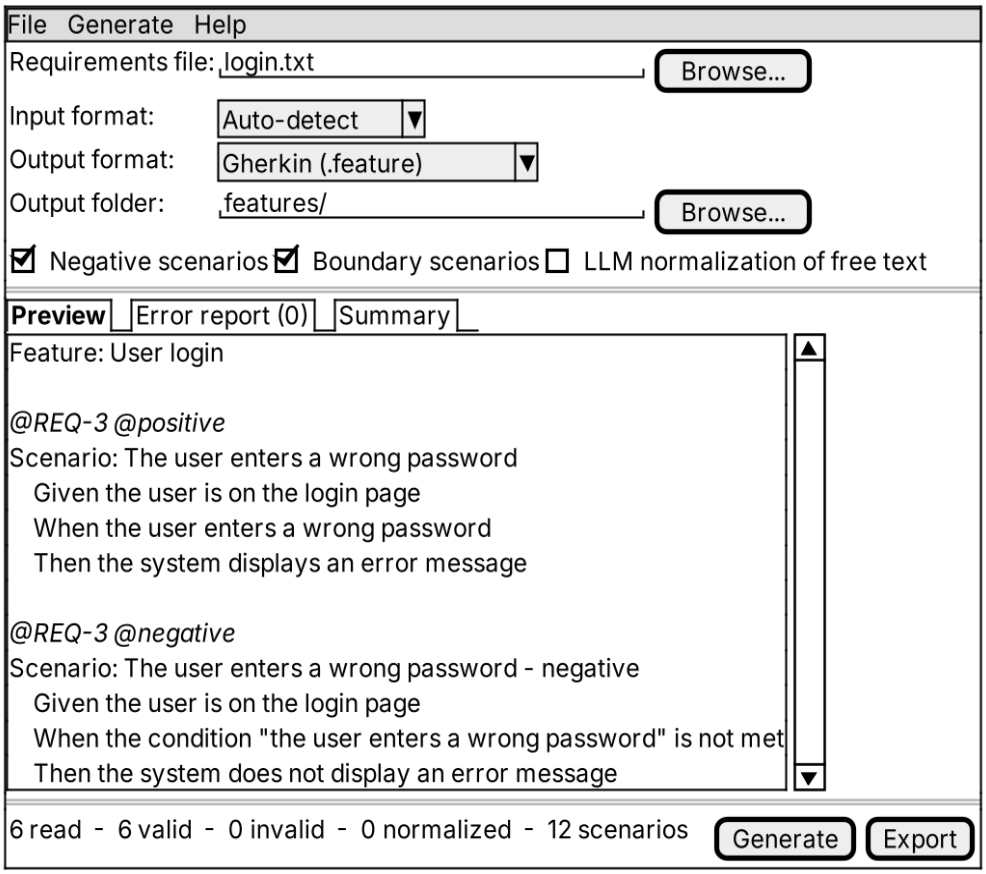


Figure 13: *Mock-up of the optional graphical interface*

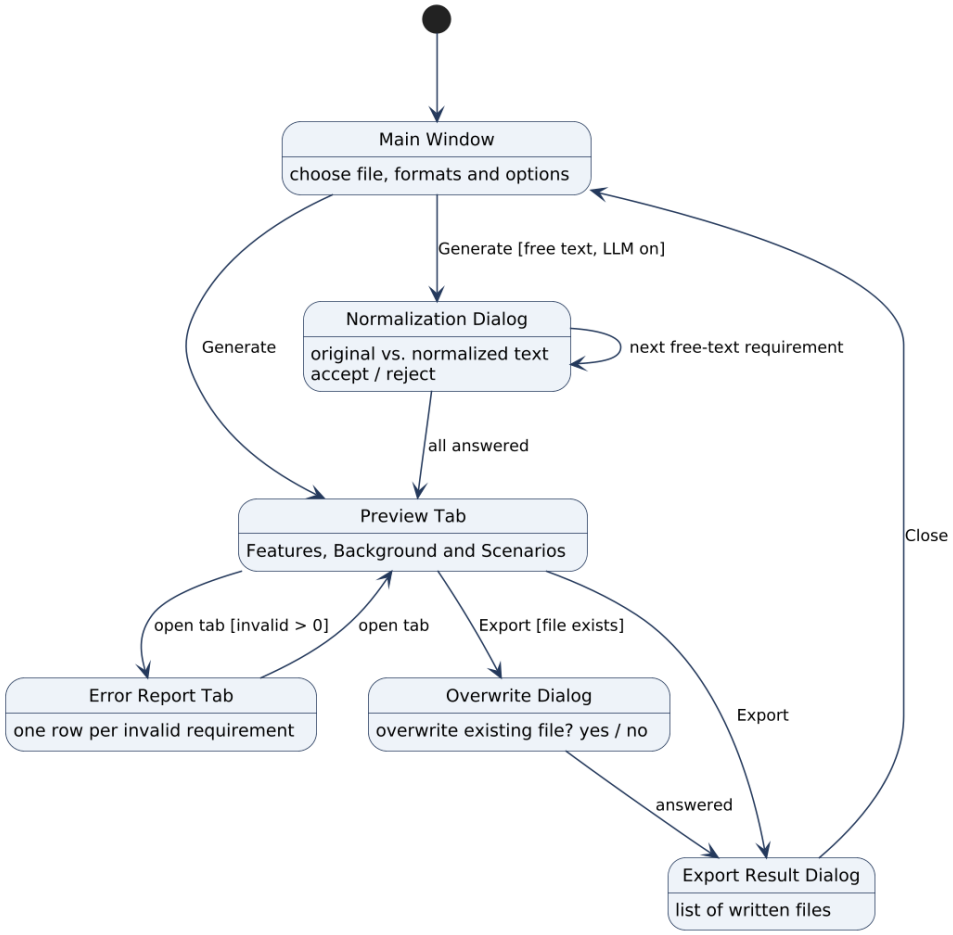


Figure 14: *Navigation path of the graphical interface*

File Generate Help

Requirements file: payment.txt [Browse...]

Preview | **Error report (2)** | Summary

Line	ID	Reason	Expected pattern / hint
4	REQ-4	missing response clause	When <trigger>, the <system> shall <response>.
			hint: write "the system shall process the payment"
9	REQ-2	duplicate requirement ID	first used on line 2

10 read - 8 valid - 2 invalid - 0 normalized - 8 scenarios [Save report...] [Generate] [Export]

Figure 15: *GUI mock-up - Error Report tab (UC-5)*

Normalize free-text requirement (1 of 2)

Line L1 - original text:

Users should get an email after they reset their password

Normalized by LLM Service (valid EARS):

When the user resets the password, the system shall send a confirmation email.

Scenarios of an accepted requirement are tagged @llm-normalized.

[Reject] [Accept]

Figure 16: *GUI mock-up - Normalization dialog (UC-9)*

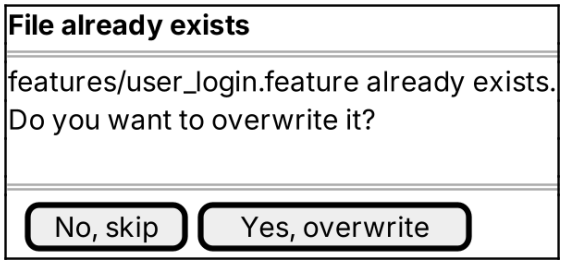


Figure 17: *GUI mock-up - Overwrite dialog (UC-7)*

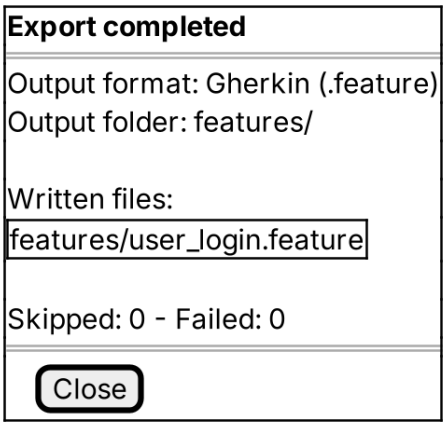


Figure 18: *GUI mock-up - Export result dialog (UC-7)*

4 Glossary of Terms

The glossary lists the terms of the application domain. The same names are used in the requirements, use cases and classes.

Table 7: *Glossary of terms*

Term	Definition
Acceptance criterion	A condition of a user story written as Given / When / Then lines; it becomes one Scenario.
Background	A Gherkin section that holds the Given steps shared by all scenarios of a feature, so that the set-up is written once.
BDD	Behavior-Driven Development: describing the expected behaviour of a system in plain language and deriving tests from these descriptions [6].
Boundary scenario	A scenario that tests the values just below, at and just above a numeric limit of a requirement; written as a Scenario Outline.
Clause	One part of an EARS sentence that starts with While, Where, When, If or shall. Each clause becomes one step.
Compound requirement	An EARS requirement with more than one "shall" clause; accepted with a warning.
Cucumber	An open-source tool that runs Gherkin scenarios as automated tests [5].
EARS	Easy Approach to Requirements Syntax: a set of fixed sentence patterns for writing requirements without ambiguity [4].
EARS pattern	One of the five EARS sentence structures: ubiquitous, event-driven, state-driven, unwanted behaviour, optional feature.
Error report	The list of invalid requirements of a generation session with line number, ID, text, reason, expected pattern and hint.
Examples table	The table of values of a Scenario Outline; the outline runs once per row.
Feature	A group of related scenarios; the top-level element of a .feature file.
.feature file	A text file that contains one Feature written in Gherkin.
Free-text requirement	A requirement that follows neither an EARS pattern nor the user story template.
Generation session	One run of the system on one requirements file with one set of formats and options.
Gherkin	The structured language for BDD scenarios with the keywords Feature, Background, Scenario, Scenario Outline, Examples, Given, When, Then and And [5].
Input format	The way requirements are written in the input file: EARS, user story or free text.
LLM Service	An external large-language-model service that rewrites free text as EARS sentences.
Negative scenario	A scenario in which the trigger or condition of a requirement is not met, so the response must not occur.
Normalization	Rewriting a free-text requirement as an equivalent EARS sentence, accepted by the Test Engineer.
Output format	The format of the generated files: Gherkin, Markdown or JSON.
Positive scenario	A scenario in which the trigger or condition of a requirement is met and the response occurs; generated for every valid requirement.
Requirement	One entry of the requirements file that describes a single required behaviour.

Term	Definition
Requirement ID	A unique identifier of a requirement (e.g., REQ-7, or L12 for line 12); used as a tag on generated scenarios.
Requirements file	The plain-text input file that contains the requirements.
Response clause	The "shall …" part of an EARS requirement; it becomes the Then step.
Scenario	One concrete example of system behaviour made of ordered steps.
Scenario Outline	A scenario template with placeholders such as <count>, filled from an examples table.
Step	One line of a scenario or background that starts with Given, When, Then or And.
Tag	A label starting with "@" written above a scenario; used for the requirement ID, the scenario kind and normalization.
Test Engineer	The human actor: a developer or tester who prepares acceptance tests.
User story	A requirement in the form "As a <role>, I want <goal>, so that <benefit>" [7].
Validation	Checking a requirement against its input format and patterns before generation.

5 Priority

Requirements are prioritised with the MoSCoW method. The table below also gives the matching level of the high / medium / low scheme of Bruegge and Dutoit [8] used in the course.

Table 8: *Meaning of the priority levels*

Priority	Meaning	Course level
Must	Required for the first working version; without it the system does not solve P1-P3. Must be demonstrated.	High - addressed in analysis, design and implementation
Should	Important and planned for the final delivery, but the system is usable without it.	Medium - addressed in analysis and design, demonstrated in the second iteration
Could	Desirable; implemented if time allows (LLM normalization, GUI, boundary outlines).	Low - addressed in analysis; implemented if time allows

Table 9: *Priority of functional requirements*

Priority	Functional requirements	Count
Must	FR-1 … FR-3, FR-8 … FR-10, FR-13 … FR-19, FR-23, FR-25, FR-26, FR-31 … FR-33, FR-37 … FR-40, FR-42, FR-43, FR-45, FR-46, FR-50 … FR-52	30
Should	FR-4 … FR-7, FR-11, FR-12, FR-20, FR-21, FR-24, FR-27, FR-28, FR-30, FR-34, FR-35, FR-44, FR-47 … FR-49, FR-58	19
Could	FR-22, FR-29, FR-36, FR-41, FR-53 … FR-57, FR-59	10

Table 10: *Priority of non-functional requirements*

Priority	Non-functional requirements	Count
Must	NFR-2, NFR-6, NFR-7, NFR-19 … NFR-21, NFR-23, NFR-24, NFR-29, NFR-32	10
Should	NFR-1, NFR-3, NFR-4, NFR-8 … NFR-10, NFR-12 … NFR-15, NFR-17, NFR-22, NFR-25, NFR-31	14
Could	NFR-5, NFR-11, NFR-16, NFR-18, NFR-26 … NFR-28, NFR-30, NFR-33	9

A use case has the highest priority of its functional requirements:

Table 11: *Priority of use cases*

Use case	Priority	Use case	Priority
UC-1 Load Requirements File	Must	UC-6 Preview Scenarios	Must
UC-2 Select Input Format	Must	UC-7 Export Scenarios	Must
UC-3 Generate Scenarios	Must	UC-8 Select Output Format	Must
UC-4 Validate Requirements	Must	UC-9 Normalize Free-Text Requirement	Should
UC-5 View Error Report	Must		

6 Traceability

6.1 Traceability to the project problem statement

Table 12 traces each statement of the problem statement given by the instructor [2] to the requirements that satisfy it.

Table 12: *Problem statement → requirements*

Statement in the problem statement [2]	Requirements
Ingests structured software feature requirements written in EARS syntax or standard user story templates.	FR-1, FR-9 … FR-11, FR-13, FR-20, FR-21
Automatically translates them into structured BDD feature files containing Gherkin scenarios (Given-When-Then).	FR-14 … FR-18, FR-25, FR-26, FR-45, NFR-24
Language parsers, requirement AST elements and scenario formatters are modeled as independent object modules.	NFR-13 … NFR-15; domain classes Clause, Step
Transformation logic decouples input requirement formats from output test syntax.	FR-51, FR-52, NFR-13, NFR-14
Builder: step-by-step construction of complex GherkinScenario objects containing multiple Given, When and Then steps.	FR-17, FR-29, FR-30, NFR-21
Adapter: translating raw requirement models into unified syntax nodes for the test generator engine.	FR-9, FR-20, FR-53, NFR-21
Chain of Responsibility: passing input text through sequential validation, tokenization and scenario-building pipelines.	FR-31 … FR-33, FR-37, NFR-15, NFR-21

6.2 Problems and objectives → use cases → requirements

Table 13: *Problems → objectives → use cases → requirements*

Problem	Objectives	Use cases	Requirements
P1 - Manual writing is slow	O-1, O-8	UC-1, UC-3, UC-7, UC-9	FR-1, FR-3, FR-13 … FR-16, FR-20, FR-21, FR-45, FR-53, NFR-10, NFR-12
P2 - Inconsistent results	O-2, O-3, O-4, O-5	UC-3, UC-4, UC-5	FR-14 … FR-19, FR-31 … FR-36, FR-38 … FR-40, NFR-4, NFR-7, NFR-24
P3 - Tests drift from requirements	O-3, O-6	UC-3, UC-6, UC-7	FR-4, FR-5, FR-23, FR-42, FR-43, FR-57, NFR-7, NFR-25
P4 - Negative and edge cases forgotten	O-7	UC-3, UC-6	FR-24, FR-28 … FR-30
Extensibility to new formats	O-9	UC-2, UC-8	FR-9, FR-51, NFR-13, NFR-14, NFR-16

6.3 Use case ↔ functional requirement matrix

Table 14 shows that every use case is linked to at least one functional requirement and that every functional requirement belongs to at least one use case.

Table 14: *Use case ↔ functional requirement traceability matrix*

FR	UC-1	UC-2	UC-3	UC-4	UC-5	UC-6	UC-7	UC-8	UC-9
FR-1	X								
FR-2	X								
FR-3	X								
FR-4	X								
FR-5	X								
FR-6	X								
FR-7	X								
FR-8	X								
FR-9		X							
FR-10		X							
FR-11		X							
FR-12		X							
FR-13			X						
FR-14			X						
FR-15			X						
FR-16			X						
FR-17			X						
FR-18			X						
FR-19			X						
FR-20			X						
FR-21			X						
FR-22			X						
FR-23			X						
FR-24			X						
FR-25			X						
FR-26			X						
FR-27			X						
FR-28			X						
FR-29			X						
FR-30			X						
FR-31			X	X					

FR	UC-1	UC-2	UC-3	UC-4	UC-5	UC-6	UC-7	UC-8	UC-9
FR-32				X					
FR-33				X					
FR-34				X					
FR-35				X					
FR-36				X					
FR-37			X	X					
FR-38					X				
FR-39					X				
FR-40					X				
FR-41					X				
FR-42						X			
FR-43						X			
FR-44						X			
FR-45							X		
FR-46							X		
FR-47							X		
FR-48							X		
FR-49							X		
FR-50							X		
FR-51								X	
FR-52								X	
FR-53									X
FR-54									X
FR-55									X
FR-56									X
FR-57									X
FR-58									X
FR-59									X
Total	8	4	20	7	4	3	6	2	7

6.4 Non-functional requirements → use cases

Table 15: *Non-functional requirements → use cases*

Category	NFRs	Applies to
Usability	NFR-1 … NFR-5	All use cases; NFR-5 only GUI
Reliability	NFR-6 … NFR-9	UC-1, UC-3, UC-4, UC-7
Performance	NFR-10 … NFR-12	UC-1, UC-3, UC-7
Supportability	NFR-13 … NFR-18	UC-2, UC-4, UC-8, UC-9
Implementation	NFR-19 … NFR-22	Whole system
Interface	NFR-23 … NFR-28	UC-1, UC-7, UC-9; NFR-28 all use cases
Packaging	NFR-29, NFR-30	Whole system
Legal	NFR-31 … NFR-33	UC-9; whole system

6.5 Requirements → domain classes

Table 16: *Requirements → domain classes*

Domain class	Requirements
GenerationSession	FR-10, FR-28, FR-29, FR-43, FR-53
RequirementsFile	FR-1 … FR-3, NFR-23
Requirement, EarsRequirement, Clause	FR-3 … FR-6, FR-13 … FR-16, FR-32, FR-33, FR-36
UserStory, AcceptanceCriterion	FR-7, FR-20 … FR-22, FR-34
FreeTextRequirement	FR-12, FR-53 … FR-56, FR-58, FR-59
ValidationError, ErrorReport	FR-32 … FR-35, FR-38 … FR-41
Feature, Background	FR-6, FR-25, FR-26, FR-30, FR-42
Scenario, ScenarioOutline, ExamplesTable, Step	FR-17 … FR-19, FR-23, FR-24, FR-27 … FR-29, FR-57
OutputFile	FR-45 … FR-52

Traceability from requirements to test cases will be added in the Software Test Document.

6.6 Summary

Table 17: *Summary of the analysis model*

Item	Count
Use cases	9
Actors	2 (Test Engineer, LLM Service)
Functional requirements	59
Non-functional requirements	33
Scenarios (visionary + as-is)	5 + 1

Item	Count
Entity classes in the domain model	17
Sequence diagrams	5
Activity diagrams	1
State machine diagrams	2 (+1 GUI navigation)
UI mock-ups	6 CLI + 5 GUI